

Retrieval-Augmented Generation (RAG): Architecture and Workflow

Technical Overview

September 2025

1 Introduction

Retrieval-Augmented Generation (RAG) combines information retrieval with large language models (LLMs) to produce accurate, contextually relevant responses. By retrieving external documents to ground LLM outputs, RAG reduces hallucinations and enhances performance in applications like question answering and knowledge bases.

2 Architecture

RAG systems consist of three core components:

- **Embedding Model:** Generates dense vector representations of text (e.g., Hugging Face's all-MiniLM-L6-v2).
- **Vector Database:** Stores embeddings for efficient similarity search (e.g., Qdrant with HNSW indexing).
- **LLM:** Generates responses using retrieved context (e.g., Llama or GPT via Hugging Face's transformers).

3 Workflow

The RAG pipeline operates as follows:

1. **Chunking:** Documents are split into smaller segments (e.g., 256–512 tokens) to preserve semantic coherence.
2. **Embedding:** Chunks are encoded into vectors using an embedding model and stored in a vector database.
3. **Retrieval:** A query is embedded, and the vector database retrieves the most similar chunks (e.g., using cosine similarity).
4. **Generation:** Retrieved chunks are combined with the query and fed to the LLM to generate a response.

Example in Python:

```
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient
from transformers import AutoTokenizer, AutoModelForCausalLM

# Embed and store
```

```
embedder = SentenceTransformer('all-MiniLM-L6-v2')
client = QdrantClient("localhost", port=6333)
docs = ["AI transforms industries."]
embeddings = embedder.encode(docs)
client.upsert("docs", points=[{"id": 1, "vector": embeddings[0].tolist(), "p

# Retrieve and generate
query = "AI impact"
query_vec = embedder.encode([query])[0]
hits = client.search("docs", query_vector=query_vec.tolist(), limit=1)
context = [hit.payload["text"] for hit in hits]
model = AutoModelForCausalLM.from_pretrained("gpt2")
tokenizer = AutoTokenizer.from_pretrained("gpt2")
prompt = f"Context: {context[0]}\nQuestion: {query}\nAnswer:"
inputs = tokenizer(prompt, return_tensors="pt")
output = model.generate(**inputs, max_length=50)
print(tokenizer.decode(output[0]))
```

4 Best Practices

- **Chunking Strategy:** Use sentence-based or overlapping chunks to maintain context.
- **Embedding Quality:** Choose models like multi-qa-MiniLM for task-specific performance.
- **Scalability:** Leverage Qdrant's quantization and async APIs for large datasets.
- **Evaluation:** Monitor retrieval precision and LLM output relevance.

5 Conclusion

RAG enhances LLMs by grounding responses in retrieved context, making it ideal for knowledge-intensive tasks. Tools like Qdrant and Hugging Face simplify implementation, enabling scalable, accurate systems. For more, see Qdrant Docs and Hugging Face Docs.